

Code Revision Questions

1. Logistic Regression – Breast Cancer Dataset

Build a Logistic Regression model to classify whether a tumor is malignant or benign using the Breast Cancer dataset, which contains measurements of cell nuclei.

The dataset includes features such as:

- mean radius
- mean texture
- mean perimeter
- mean area
- mean smoothness

The target labels are:

- 0 = malignant
- 1 = benign

Use `load_breast_cancer()` from `scikit-learn`, which provides:

- `data.data` → features
- `data.target` → labels

Your tasks are to:

- Split the dataset into training and testing sets
- Implement Logistic Regression
- Train the model
- Evaluate the model using Accuracy and Confusion Matrix
- Explain each step of the code with clear comments

Expected Output

Code for:

- loading data
- preprocessing

model training
performance evaluation

with correct syntax and indentation

Sample Answer:

Import necessary libraries

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Import Libraries: These libraries are used to load the dataset, split data, build the Logistic Regression model, and evaluate performance.

Load the Breast Cancer dataset

```
data = load_breast_cancer()
X = data.data # Features
y = data.target # Target: 0 = malignant, 1 = benign
```

Load Dataset: The features of tumors are stored in X and the target labels (malignant or benign) are stored in y.

Split the dataset into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Split Dataset: 80% of data is used for training and 20% for testing to evaluate the model fairly.

Create Logistic Regression model

```
model = LogisticRegression(max_iter=1000)
```

Create Model: A Logistic Regression model object is created.

Train the model

```
model.fit(X_train, y_train)
```

Train Model: The model learns from the training data.

Make predictions on the test set

```
y_pred = model.predict(X_test)
```

Predictions: The trained model predicts labels for unseen test data.

Evaluate the model

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Evaluation: Accuracy, confusion matrix, and classification report are printed to assess model performance.

2. K-Nearest Neighbors (KNN) Classifier – Iris Dataset

Build a K-Nearest Neighbors (KNN) classification model using the Iris dataset, which contains measurements of iris flowers.

The dataset includes features such as:

sepal length

sepal width

petal length

petal width

The target labels are:

Setosa - 0

Versicolor - 1

Virginica - 2

Use `load_iris()` from `scikit-learn`, which provides:

`data.data` → features

`data.target` → labels

Your tasks are to:

Split the dataset into training and testing sets

Standardize the training and testing datasets

Implement KNN classifier with $k = 5$

Train the model

Evaluate the model using Accuracy

Explain each step of the code with clear comments

Expected Output

Code for:

loading data

preprocessing

model training

performance evaluation

with correct syntax and indentation

Sample Answer:

Import necessary libraries

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

Load the Iris dataset

```
iris = load_iris()
X = iris.data # Features
y = iris.target # Target labels
```

Split the dataset

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Standardize features

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Create KNN classifier with k = 5

```
model = KNeighborsClassifier(n_neighbors=5)
```

Train model

```
model.fit(X_train, y_train)
```

Predict

```
y_pred = model.predict(X_test)
```

Evaluate

```
accuracy = accuracy_score(y_test, y_pred)  
print("Accuracy:", accuracy)
```

3. K-Nearest Neighbors (KNN) Regressor – Diabetes Dataset

Build a K-Nearest Neighbors (KNN) regression model to predict diabetes progression using the Diabetes dataset.

The target variable represents a quantitative measure of disease progression one year after baseline.

Use `load_diabetes()` from scikit-learn, which provides features and target values.

Your tasks are to:

- Split the dataset into training and testing sets
- Standardize the training and testing datasets
- Implement KNN Regressor
- Train the model
- Evaluate the model using Mean Squared Error (MSE)
- Explain each step of the code with clear comments

Expected Output

Code for:

loading data
preprocessing
model training
performance evaluation

with correct syntax and indentation

4. Decision Tree Classifier – Wine Dataset

Build a Decision Tree classification model using the Wine dataset, which contains results of chemical analysis of wines.

The dataset includes features related to chemical composition of wine samples.

The target labels are different wine classes.

Use `load_wine()` from `scikit-learn`, which provides:

`data.data` → features

`data.target` → labels

Your tasks are to:

Split the dataset into training and testing sets

Implement Decision Tree Classifier

Train the model

Evaluate the model using Accuracy

Explain each step of the code with clear comments

Expected Output

Code for:

loading data
preprocessing
model training
performance evaluation

with correct syntax and indentation

Sample Answer:

Import libraries

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

Load dataset

```
wine = load_wine()
X = wine.data
y = wine.target
```

Split dataset

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Create Decision Tree model

```
model = DecisionTreeClassifier()
```

Train

```
model.fit(X_train, y_train)
```

Predict

```
y_pred = model.predict(X_test)
```

Evaluate

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

5. Decision Tree Regressor – Diabetes Dataset

Build a Decision Tree regression model to predict diabetes progression using the Diabetes dataset.

The target variable is a quantitative measure of disease progression one year after baseline.

Your tasks are to:

Split the dataset into training and testing sets

Implement Decision Tree Regressor

Train the model

Evaluate the model using Mean Squared Error (MSE) and R-squared

Explain each step of the code with clear comments

Expected Output

Code for:

loading data

preprocessing

model training

performance evaluation

with correct syntax and indentation

6. Support Vector Machine (SVM) Classifier – Iris Dataset

Build a Support Vector Machine (SVM) classification model using the Iris dataset.

The dataset includes flower measurements such as sepal length, sepal width, petal length, and petal width.

The target labels are:

Setosa

Versicolor

Virginica

Your tasks are to:

Split the dataset into training and testing sets

Standardize the training and testing datasets

Implement SVM with a linear kernel

Train the model

Evaluate the model using Accuracy

Explain each step of the code with clear comments

Expected Output

Code for:

loading data

preprocessing

model training

performance evaluation

with correct syntax and indentation

7. Support Vector Regression (SVR) – Diabetes Dataset

Build a Support Vector Regression (SVR) model to predict diabetes progression using the Diabetes dataset.

The target variable represents a numerical measure of disease progression.

Your tasks are to:

Split the dataset into training and testing sets

Standardize the training and testing datasets

Implement SVR

Train the model

Evaluate the model using Mean Squared Error (MSE)

Explain each step of the code with clear comments

Expected Output

Code for:

loading data

preprocessing

model training

performance evaluation

with correct syntax and indentation

8. Naive Bayes Classifier – Breast Cancer Dataset

Build a Gaussian Naive Bayes classification model using the Breast Cancer dataset.

The target variable indicates whether the tumor is malignant or benign.

Your tasks are to:

Split the dataset into training and testing sets

Implement Gaussian Naive Bayes

Train the model

Evaluate the model using Accuracy

Explain each step of the code with clear comments

Expected Output

Code for:

loading data

preprocessing

model training

performance evaluation

with correct syntax and indentation

9. Random Forest Classifier – Wine Dataset

Build a Random Forest classification model using the Wine dataset.

The dataset includes features from chemical analysis of wines.

The target labels represent the wine classes.

Your tasks are to:

Split the dataset into training and testing sets

Implement Random Forest Classifier

Train the model

Evaluate the model using Accuracy

Explain each step of the code with clear comments

Expected Output

Code for:

loading data

preprocessing

model training

performance evaluation

with correct syntax and indentation

10. Random Forest Regressor – Diabetes Dataset

Build a Random Forest regression model to predict diabetes progression using the Diabetes dataset.

The target variable represents disease progression one year after baseline.

Your tasks are to:

Split the dataset into training and testing sets

Implement Random Forest Regressor

Train the model

Evaluate the model using Mean Squared Error (MSE) and R-squared

Explain each step of the code with clear comments

Expected Output

Code for:

loading data

preprocessing

model training

performance evaluation

with correct syntax and indentation

11. Ridge Regression – Diabetes Dataset

Build a Ridge Regression model to predict diabetes progression using the Diabetes dataset.

The target variable is a continuous measure of disease progression.

Your tasks are to:

Split the dataset into training and testing sets

Standardize the training and testing datasets

Implement Ridge Regression

Train the model

Evaluate the model using Mean Squared Error (MSE) and R-squared

Explain each step of the code with clear comments

Expected Output

Code for:

loading data

preprocessing

model training

performance evaluation

with correct syntax and indentation

12. Lasso Regression – Diabetes Dataset

Build a Lasso Regression model to predict diabetes progression using the Diabetes dataset.

Your tasks are to:

Split the dataset into training and testing sets

Standardize the training and testing datasets

Implement Lasso Regression

Train the model

Evaluate the model using Mean Squared Error (MSE) and R-squared

Explain each step of the code with clear comments

Expected Output

Code for:

loading data

preprocessing

model training

performance evaluation

with correct syntax and indentation

13. Elastic Net Regression – Diabetes Dataset

Build an Elastic Net Regression model to predict diabetes progression using the Diabetes dataset.

Your tasks are to:

Split the dataset into training and testing sets

Standardize the training and testing datasets

Implement Elastic Net Regression

Train the model

Evaluate the model using Mean Squared Error (MSE) and R-squared

Explain each step of the code with clear comments

Expected Output

Code for:

loading data

preprocessing

model training

performance evaluation

with correct syntax and indentation

14. Principal Component Analysis (PCA) with Logistic Regression – Breast Cancer Dataset

Build a Logistic Regression model using Principal Component Analysis (PCA) on the Breast Cancer dataset.

The target variable indicates whether the tumor is malignant or benign.

Your tasks are to:

Split the dataset into training and testing sets

Standardize the training and testing datasets

Apply PCA for dimensionality reduction

Train Logistic Regression

Evaluate the model using Accuracy

Explain each step of the code with clear comments

Expected Output

Code for:

loading data

preprocessing

dimensionality reduction

model training

performance evaluation

with correct syntax and indentation

15. Linear Regression – California Housing Dataset

Build a Linear Regression model to predict house prices using the California Housing dataset.

The target variable represents median house value.

Your tasks are to:

Split the dataset into training and testing sets

Standardize the training and testing datasets

Implement Linear Regression

Train the model

Evaluate the model using Mean Squared Error (MSE) and R-squared

Explain each step of the code with clear comments

Expected Output

Code for:

loading data

preprocessing

model training

performance evaluation

with correct syntax and indentation

16. Logistic Regression – Iris Dataset

Build a Logistic Regression model to classify whether an iris flower is Setosa using the Iris dataset, which contains measurements of sepal length, sepal width, petal length, and petal width.

The target labels are:

Setosa - 0

Versicolor - 1

Virginica - 2

Your tasks are to:

Convert the problem into binary classification (Setosa vs non-Setosa)

Split the dataset into training and testing sets

Implement Logistic Regression

Train the model

Evaluate the model using Accuracy

Explain each step of the code with clear comments

Expected Output

Code for:

loading data

preprocessing

binary label conversion

model training

performance evaluation

with correct syntax and indentation

Sample Answer:

```
# Import necessary libraries
```

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn import datasets
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

```
# Load the Iris dataset
```

```
iris = datasets.load_iris()
```

```
X = iris.data # Features: sepal length, sepal width, petal length, petal width
```

```
y = iris.target # Target labels: 0 (Setosa), 1 (Versicolor), 2 (Virginica)

# Convert to a binary classification (Setosa vs. non-Setosa)
y_binary = np.where(y == 0, 1, 0) # Setosa (0) is now labeled as 1, others as 0

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y_binary, test_size=0.2,
random_state=42)

# Create a Logistic Regression model
model = LogisticRegression()

# Train the model
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)

# Evaluate the model's accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

# Display the classification report and confusion matrix
print("Classification Report:\n", classification_report(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```